

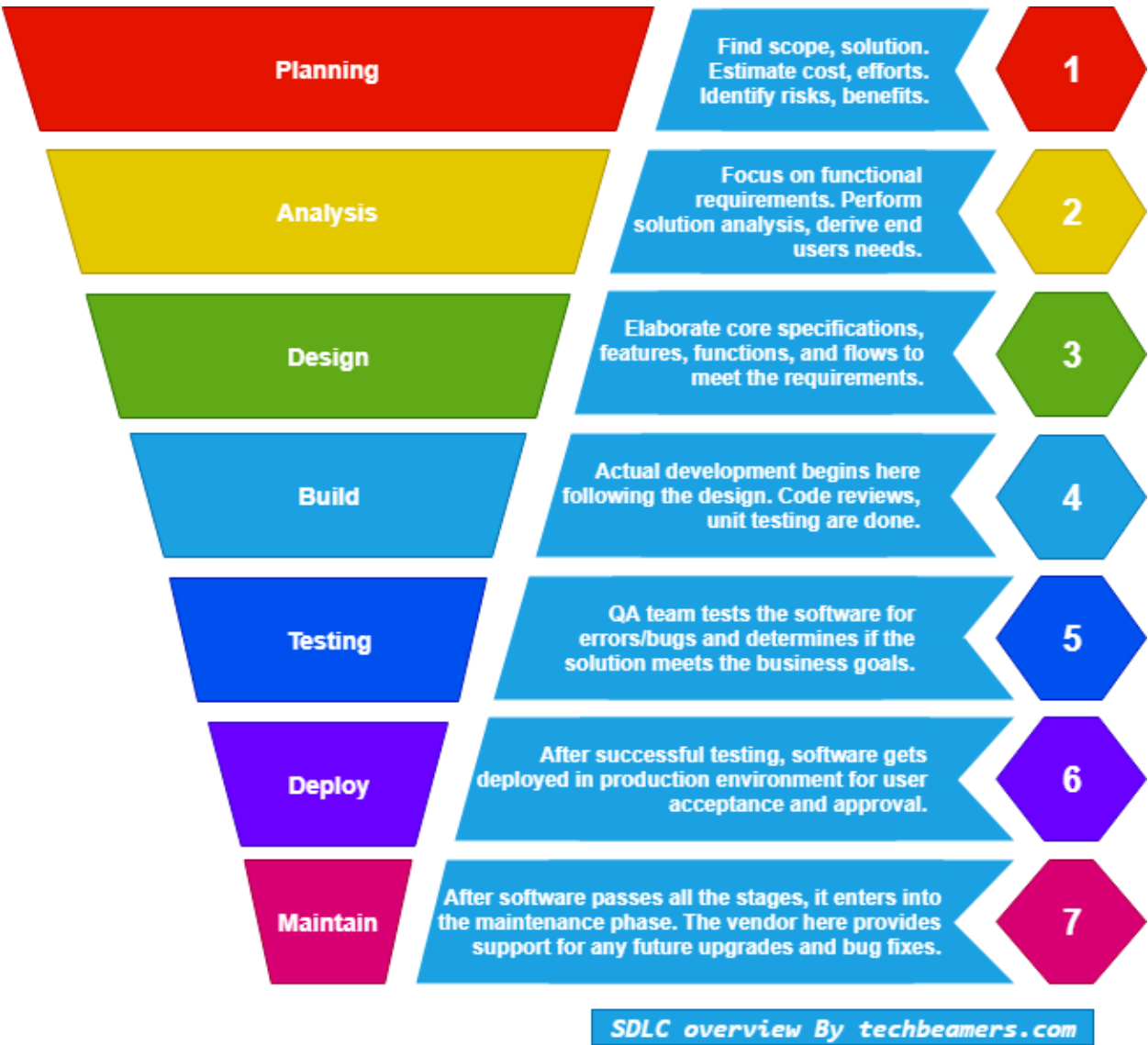
Software Development Methodologies

Software development methodologies provide structured approaches for planning, designing, developing, testing, and maintaining software systems.

SDLC (Software Development Life Cycle)

The SDLC is a systematic process used to develop high-quality software from planning to maintenance.

Phases of SDLC



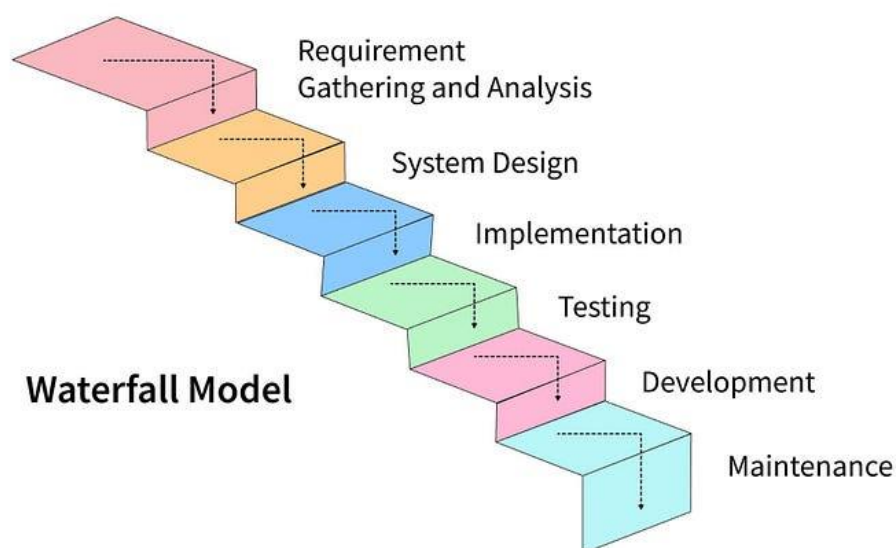
Example

Developing a Student Information System following all SDLC phases.

TYPES OF SDLC MODEL

The **Software Development Life Cycle (SDLC) models** are different approaches used to plan, develop, test, and maintain software. The most common SDLC models are:

SDLC Model	Description	Best Used For
1. Waterfall Model	A linear, sequential approach where each phase must be completed before the next begins.	Projects with stable and well-defined requirements.



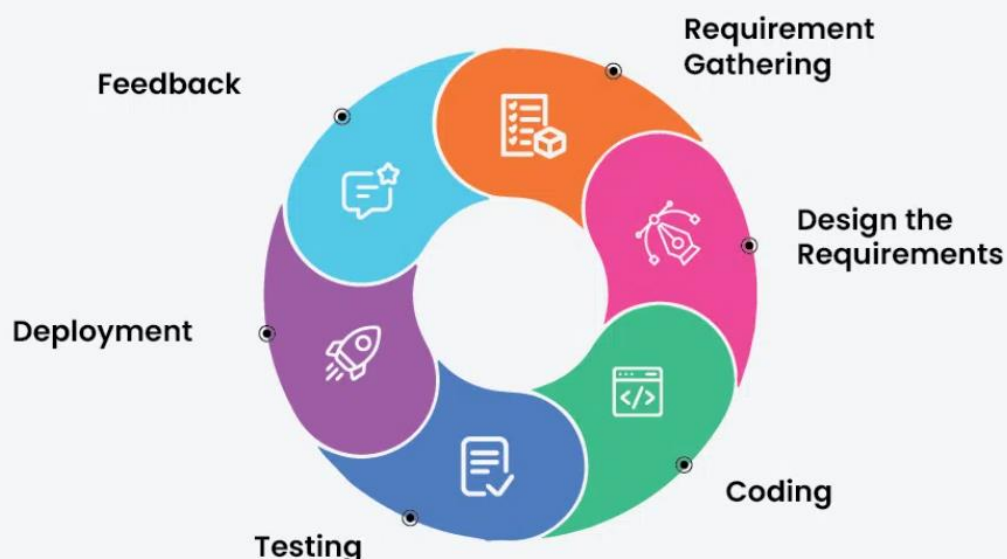
The Waterfall model is one of the oldest and most well-known models of software development.

2. Agile Model

An iterative and flexible approach where development is done in small increments (sprints) with continuous customer feedback.

Projects with changing requirements and frequent updates.

Phases of Agile SDLC

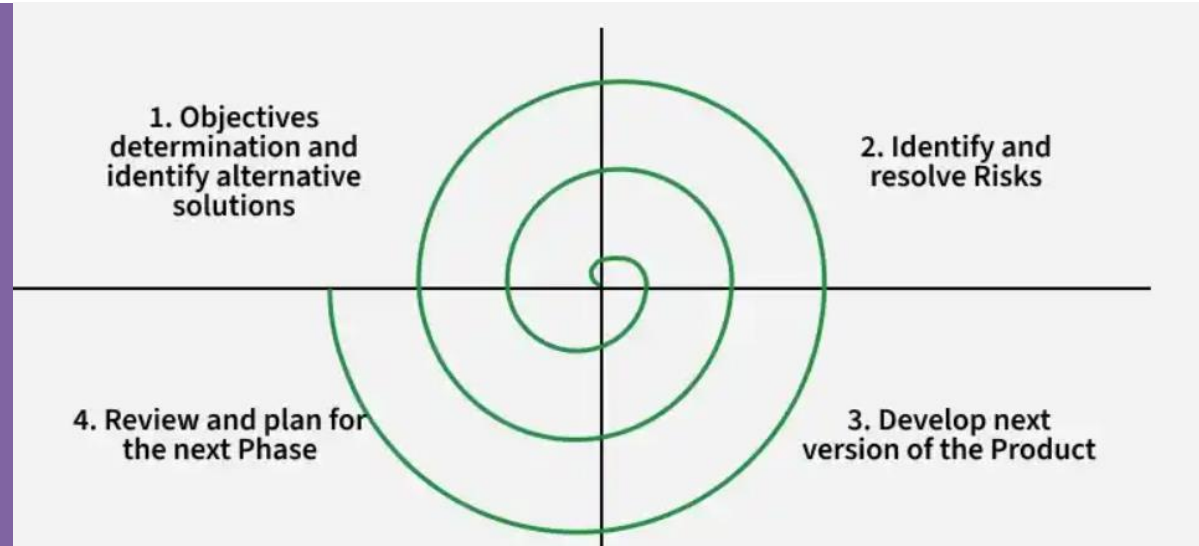


The Agile model is an iterative project management approach that breaks large projects into small, manageable development cycles called "sprints" (typically lasting 14–30 days). It emphasizes continuous testing, cross-team collaboration, and adaptability, allowing teams to deliver working software quickly and pivot easily based on customer feedback.

3. Spiral Model

Combines iterative development with risk analysis. Each cycle focuses on planning, risk assessment, development, and evaluation.

Large, high-risk, and complex systems.



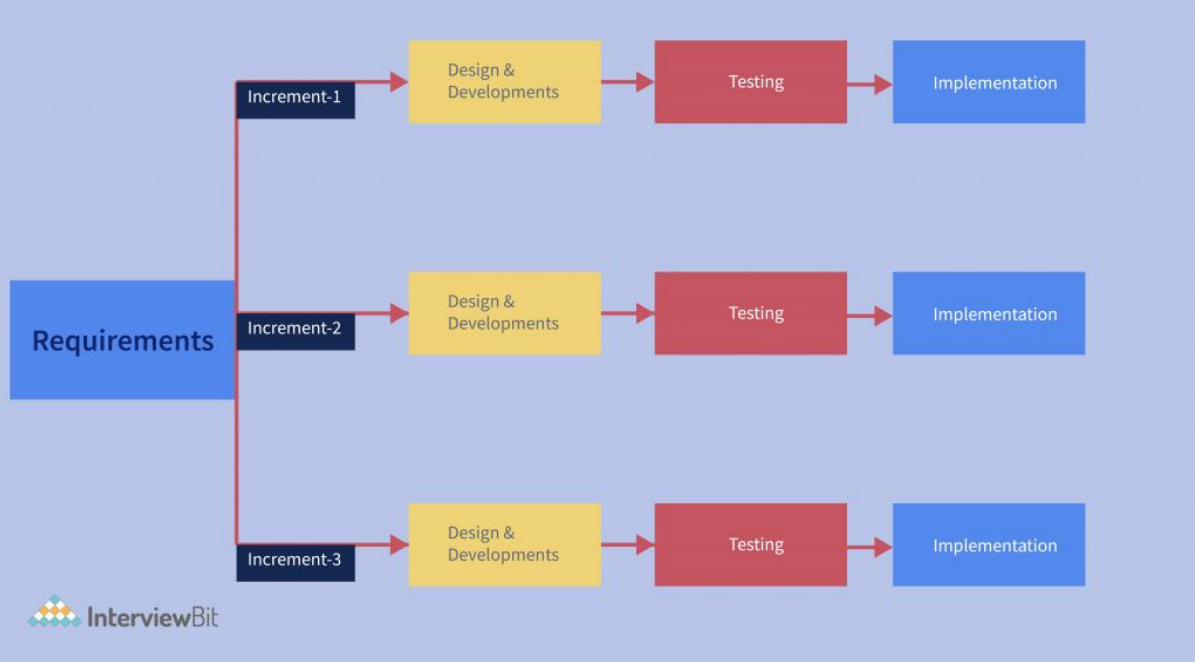
The development process is represented as a coil with many loops, where each loop or "spiral" represents a phase in the development lifecycle. As the project iterates, the spiral expands outward, increasing the scope and complexity of the software.

4. Iterative Model	The software is developed in repeated cycles, gradually adding features and improvements.	Projects where requirements may evolve over time.
--------------------	---	---



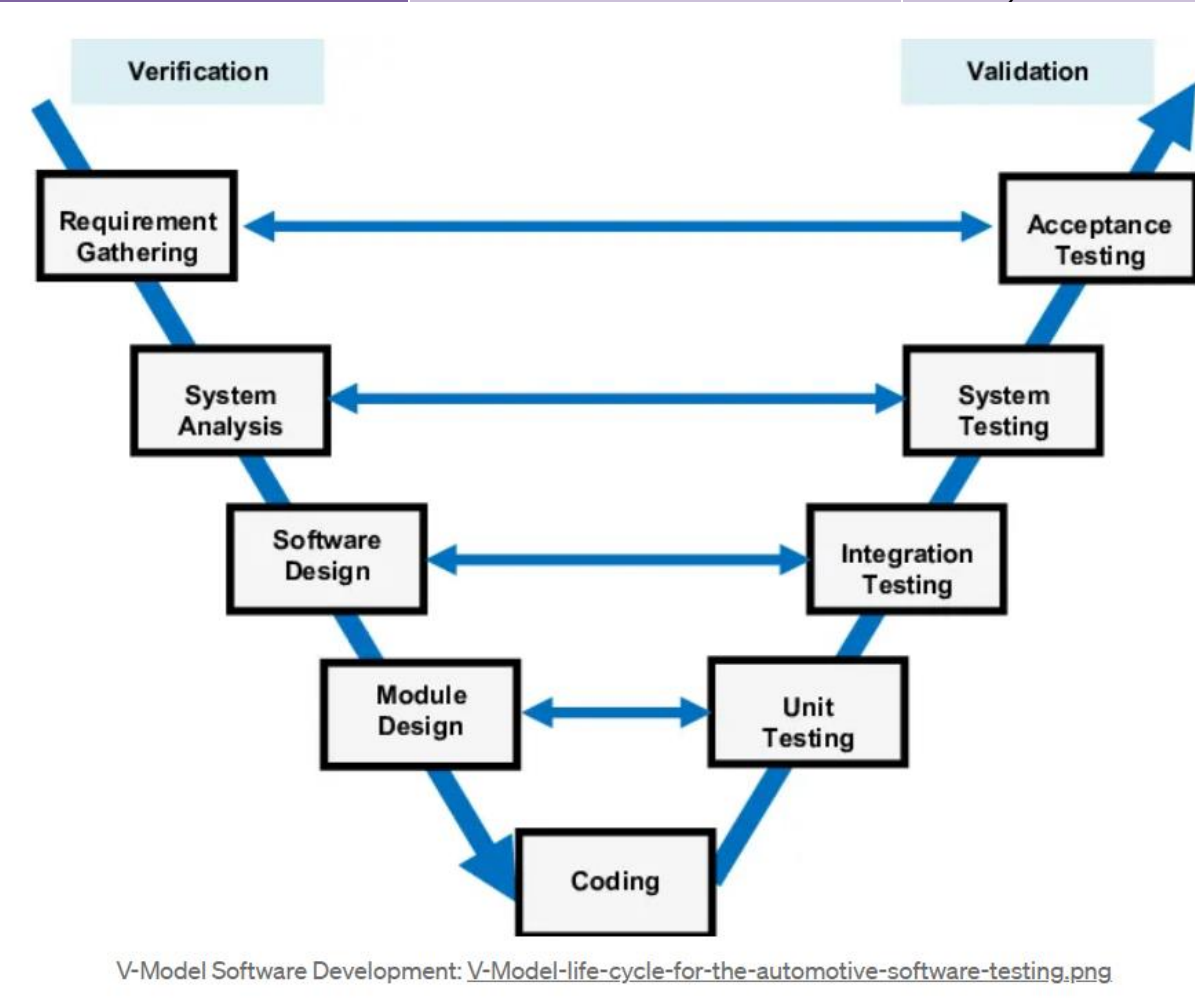
The iterative model is a software development life cycle (SDLC) approach that creates a product through repeated, cyclical phases. Rather than waiting for complete requirements, developers build a basic version (or prototype) of the system, gather stakeholder feedback, and progressively refine the software in subsequent cycles until the final system is perfected.

5. Incremental Model	The system is built and delivered in small functional modules (increments).	Projects requiring early deployment of usable features.
----------------------	---	---



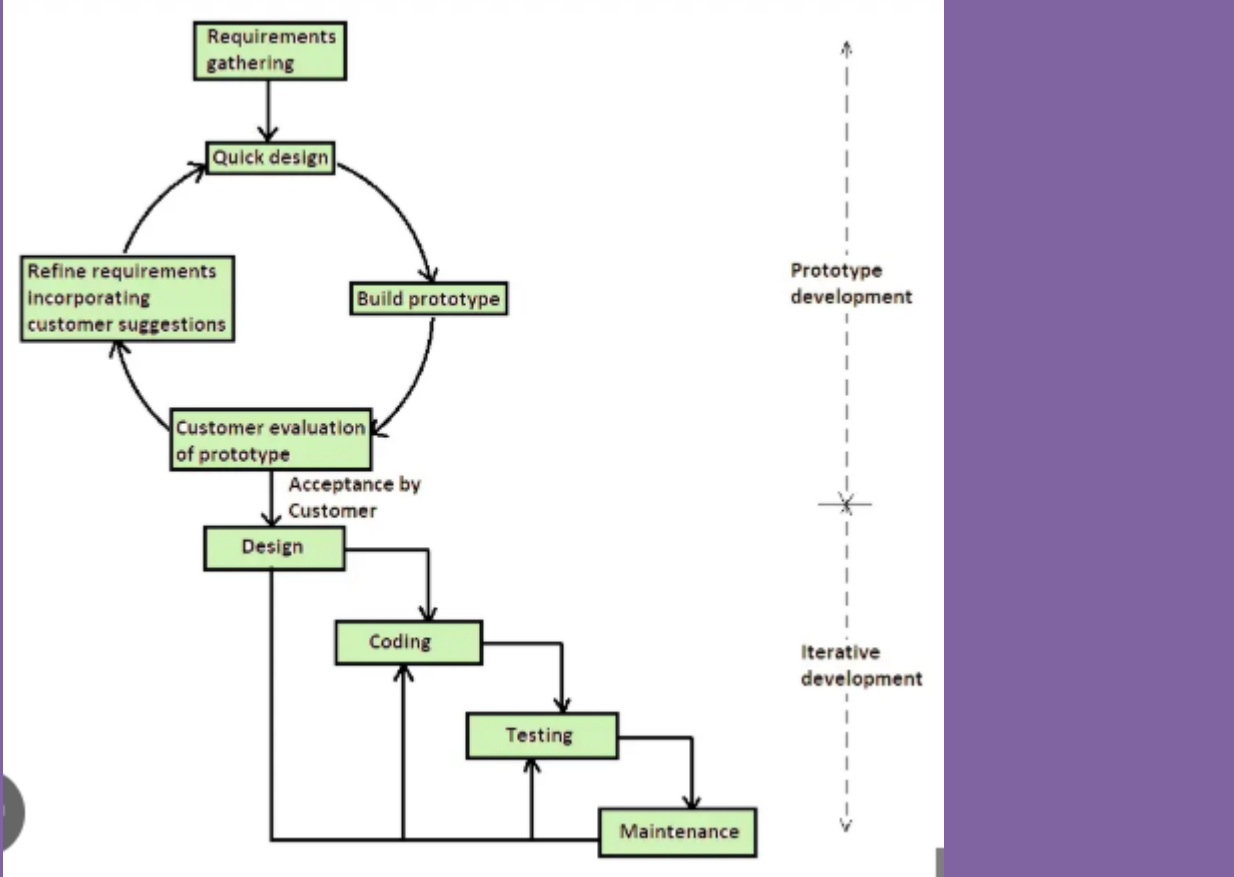
An incremental model is a software development methodology where a product is designed, implemented, and tested progressively in small, manageable parts or modules. Each release adds functional capabilities to the previous one until the final, complete system is built.

6. V-Model (Verification and Validation Model)	Similar to Waterfall, but each development phase has a corresponding testing phase.	Safety-critical systems requiring extensive testing (e.g., medical, aviation).
--	---	--



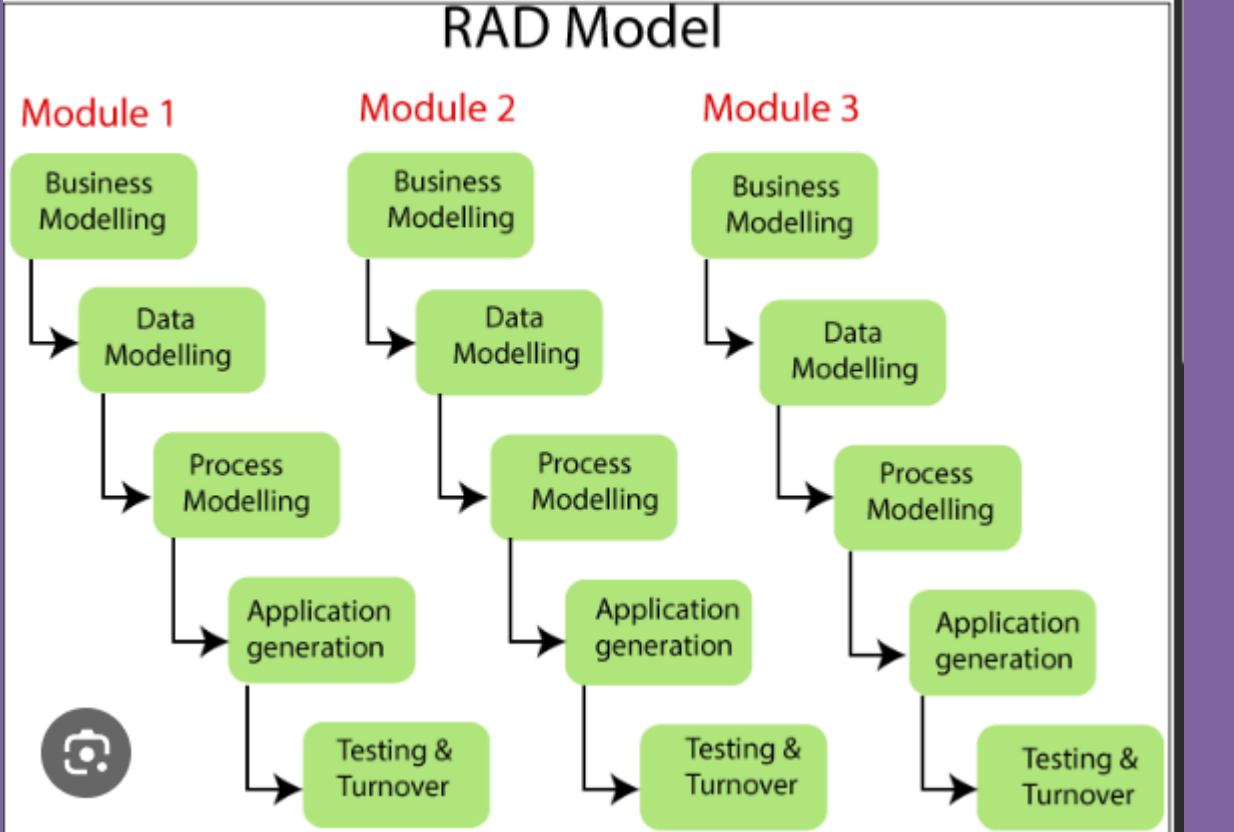
The V-Model (Verification and Validation Model) is a disciplined SDLC methodology where each phase of development on the left (e.g., requirements, design, coding) links to a corresponding testing phase on the right (e.g., unit, integration, acceptance testing). It emphasizes early defect prevention and thorough documentation.

7. Prototype Model	A prototype is developed first to gather user feedback before building the final system.	Projects with unclear or evolving user requirements.
--------------------	--	--



This model is used when the customers do not know the exact project requirements beforehand. In this model, a prototype of the end product is first developed, tested, and refined as per customer feedback repeatedly till a final acceptable prototype is achieved which forms the basis for developing the final product.

8. RAD (Rapid Application Development) Model	Focuses on rapid prototyping and fast development using reusable components.	Small to medium-sized projects with tight deadlines.
--	--	--



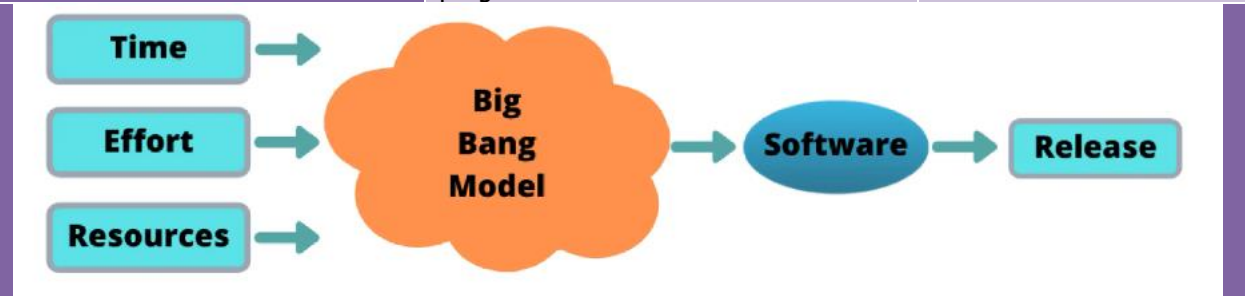
The Rapid Application Development (RAD) model is an adaptive software lifecycle approach that prioritizes rapid prototyping, iterative development, and frequent user feedback over long, upfront planning. Development happens in fast, time-boxed cycles, allowing teams to deliver working software quickly and easily adjust to changing requirements.

9. DevOps Model	Integrates software development and IT operations to enable continuous integration, testing, and deployment.	Cloud-based applications and systems requiring frequent releases.
-----------------	--	---



The DevOps model is a software engineering approach that unifies development (Dev) and operations (Ops) into a single, continuous lifecycle. It bridges organizational silos through automation, cultural collaboration, and shared responsibility, enabling teams to deliver applications and services at high velocity and reliability.

10. Big Bang Model	Minimal planning; development starts immediately and evolves as work progresses.	Small experimental or research projects.
--------------------	--	--



The Big Bang model is a start-from-scratch SDLC model where very little time is spent on planning, and we do not follow any specific process. It is the most basic SDLC model because it does not necessitate any planning. Even the client is unsure of what he wants, and the requirements are applied on the fly with little thought. The fundamental purpose of this strategy is to begin coding immediately, without following any set structure, and to provide the product to the customer. Without knowing the end result, day-to-day development begins with some preliminary prerequisites. Then, the client communicates with the development team to monitor the outcome of the development. If the end outcome is as intended, the product is approved; otherwise, another strategy is sought and worked on to achieve the desired result.

Recommended Learning Resources

- **Microsoft Learn (ASP.NET & .NET):** <https://learn.microsoft.com/dotnet/>
- **Git Documentation:** <https://git-scm.com/doc>
- **GitHub Skills:** <https://skills.github.com/>
- **Atlassian Git Tutorials:** <https://www.atlassian.com/git/tutorials>
- **Postman Learning Center (APIs):** <https://learning.postman.com/>
- **Lucidchart (Diagrams):** <https://www.lucidchart.com/pages/>
- **Draw.io (Free Diagram Tool):** <https://app.diagrams.net/>
- **MySQL Documentation:** <https://dev.mysql.com/doc/>